Artificial Intelligence Lab Record

Name: **D. Vishnu Vardhan Reddy**

Register Number: **192325039**

Index

Experiment 1 – 8-Puzzle Problem

Experiment 2 – 8-Queens Problem

Experiment 3 – Water Jug Problem

Experiment 4 – Crypt-Arithmetic Problem

Experiment 5 – Missionaries and Cannibals Problem

Experiment 1: 8-Puzzle Problem

Aim

To write a Python program that solves the 8-Puzzle. The program starts from a scrambled 3x3 board and slides the blank tile around until the tiles are in the order 1 2 3 / 4 5 6 / 7 8 _, using Breadth-First Search (BFS).

Algorithm

1. Take the starting board and first check that it can be solved. Count the inversions among the numbered tiles; if the count is odd, the puzzle has no solution and we stop.
2. Put the starting board in a queue and note it as visited.
3. Take the board at the front of the queue. If it matches the goal board, we are done and we print the moves taken to reach it.
4. Otherwise, try moving the blank Up, Down, Left and Right. A move is allowed only if the blank stays inside the 3x3 grid.
5. Every new board that has not been seen before goes to the back of the queue along with the moves used to get there.
6. Repeat from step 3. Since BFS tries shorter paths first, the answer it finds uses the fewest possible moves.

Experiment 1: 8-Puzzle Problem – Program

```
from collections import deque

GOAL = (1, 2, 3, 4, 5, 6, 7, 8, 0)


def get_neighbors(state):
    """Return list of (move, new_state) by sliding the blank (0)."""
    idx = state.index(0)
    row, col = divmod(idx, 3)
    moves = {"Up": (-1, 0), "Down": (1, 0), "Left": (0, -1), "Right": (0, 1)}
    result = []
    for name, (dr, dc) in moves.items():
        nr, nc = row + dr, col + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            nidx = nr * 3 + nc
            lst = list(state)
            lst[idx], lst[nidx] = lst[nidx], lst[idx]
            result.append((name, tuple(lst)))
    return result


def is_solvable(state):
    nums = [x for x in state if x != 0]
    inversions = sum(1 for i in range(len(nums))
                     for j in range(i + 1, len(nums)) if nums[i] > nums[j])
    return inversions % 2 == 0


def bfs(start):
    queue = deque([(start, [])])
    visited = {start}
    while queue:
        state, path = queue.popleft()
        if state == GOAL:
            return path
        for move, nxt in get_neighbors(state):
            if nxt not in visited:
                visited.add(nxt)
                queue.append((nxt, path + [(move, nxt)]))
    return None


def print_state(state):
    for i in range(0, 9, 3):
        print(" ".join(str(x) if x != 0 else "_" for x in state[i:i + 3]))
    print()


if __name__ == "__main__":
    start = (4, 1, 3, 7, 2, 6, 0, 5, 8)
    print("Initial State:")
    print_state(start)
    if not is_solvable(start):
        print("This puzzle is not solvable.")
    else:
        solution = bfs(start)
        print("Solved in", len(solution), "moves\n")
        for step, (move, state) in enumerate(solution, 1):
            print("Step", step, "- Move blank", move)
            print_state(state)
```

Experiment 1: 8-Puzzle Problem – Output

D. Vishnu Vardhan Reddy - 192325039 - p1_8puzzle.py

```
$ python3 p1_8puzzle.py

Initial State:
4 1 3
7 2 6
_ 5 8

Solved in 6 moves

Step 1 - Move blank Up
4 1 3
_ 2 6
7 5 8

Step 2 - Move blank Up
_ 1 3
4 2 6
7 5 8

Step 3 - Move blank Right
1 _ 3
4 2 6
7 5 8

Step 4 - Move blank Down
1 2 3
4 _ 6
7 5 8

Step 5 - Move blank Down
1 2 3
4 5 6
7 _ 8

Step 6 - Move blank Right
1 2 3
4 5 6
7 8 _

$
```

Result: The puzzle was solved in 6 moves, which is the shortest solution for this starting board.

Experiment 2: 8-Queens Problem

Aim

To write a Python program that places 8 queens on an 8x8 chessboard so that no two of them can attack each other, using backtracking.

Algorithm

1. Store the board as a list where the value at index r is the column of the queen sitting in row r.
2. Start with row 0 and try each column one by one.
3. Before placing a queen, check that no earlier queen is in the same column or on the same diagonal. If the square is safe, place the queen and move on to the next row.
4. If none of the columns in a row work, go back to the previous row (backtrack), lift that queen and try its next column.
5. When all 8 rows have a queen, save the arrangement as one solution.
6. Keep going until every possibility has been tried, then print how many solutions were found along with the first one.

Experiment 2: 8-Queens Problem – Program

```
N = 8


def is_safe(board, row, col):
    """board[r] = column of the queen placed in row r."""
    for r in range(row):
        c = board[r]
        if c == col or abs(c - col) == abs(r - row):
            return False
    return True


def solve(board, row, solutions):
    if row == N:
        solutions.append(board[:])
        return
    for col in range(N):
        if is_safe(board, row, col):
            board[row] = col
            solve(board, row + 1, solutions)
            board[row] = -1  # backtrack


def print_board(board):
    for r in range(N):
        print(" ".join("Q" if board[r] == c else "." for c in range(N)))
    print()


if __name__ == "__main__":
    solutions = []
    solve([-1] * N, 0, solutions)
    print("Total number of solutions for", N, "queens:", len(solutions))
    print("\nFirst solution:")
    print_board(solutions[0])
    print("Column positions (row 0..7):", solutions[0])
```

Experiment 2: 8-Queens Problem – Output

D. Vishnu Vardhan Reddy - 192325039 - p2_8queens.py

```
$ python3 p2_8queens.py

Total number of solutions for 8 queens: 92

First solution:
Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .

Column positions (row 0..7): [0, 4, 7, 5, 2, 6, 1, 3]

$
```

Result: The program found all 92 solutions to the 8-Queens problem and printed the first one as a board.

Experiment 3: Water Jug Problem

Aim

To write a Python program that measures exactly 2 litres of water using only a 4-litre jug and a 3-litre jug, searching the possible states with BFS.

Algorithm

1. Describe each situation as a pair (x, y), where x is the water in the 4-litre jug and y is the water in the 3-litre jug. Start from (0, 0).
2. From any state we can do six things: fill the 4L jug, fill the 3L jug, empty the 4L jug, empty the 3L jug, pour 4L into 3L, or pour 3L into 4L. When pouring, stop as soon as the receiving jug is full or the other jug is empty.
3. Add the start state to a queue and mark it as visited.
4. Take a state from the queue. If either jug holds the target amount, print the path that led to it.
5. Apply all six operations and add every state we have not seen before to the queue.
6. Carry on until the target shows up. If the queue becomes empty first, there is no solution.

Experiment 3: Water Jug Problem – Program

```
from collections import deque


def water_jug(cap_x, cap_y, target):
    """BFS over states (x, y) = water in jug X and jug Y."""
    start = (0, 0)
    queue = deque([(start, [start])])
    visited = {start}
    while queue:
        (x, y), path = queue.popleft()
        if x == target or y == target:
            return path
        pour_xy = min(x, cap_y - y)
        pour_yx = min(y, cap_x - x)
        next_states = [
            (cap_x, y),             # fill X
            (x, cap_y),             # fill Y
            (0, y),                 # empty X
            (x, 0),                 # empty Y
            (x - pour_xy, y + pour_xy),   # pour X -> Y
            (x + pour_yx, y - pour_yx),   # pour Y -> X
        ]
        for s in next_states:
            if s not in visited:
                visited.add(s)
                queue.append((s, path + [s]))
    return None


if __name__ == "__main__":
    cap_x, cap_y, target = 4, 3, 2
    print("Jug X capacity =", cap_x, ", Jug Y capacity =", cap_y,
          ", Target =", target)
    path = water_jug(cap_x, cap_y, target)
    if path is None:
        print("No solution possible.")
    else:
        print("Solution found in", len(path) - 1, "steps:\n")
        for i, (x, y) in enumerate(path):
            print("Step", i, ": (X =", x, ", Y =", y, ")")
```

Experiment 3: Water Jug Problem – Output

D. Vishnu Vardhan Reddy - 192325039 - p3_waterjug.py

```
$ python3 p3_waterjug.py

Jug X capacity = 4 , Jug Y capacity = 3 , Target = 2
Solution found in 4 steps:

Step 0 : (X = 0 , Y = 0 )
Step 1 : (X = 0 , Y = 3 )
Step 2 : (X = 3 , Y = 0 )
Step 3 : (X = 3 , Y = 3 )
Step 4 : (X = 4 , Y = 2 )

$
```

Result: The target of 2 litres was reached in 4 steps, ending at (X = 4, Y = 2).

Experiment 4: Crypt-Arithmetic Problem

Aim

To write a Python program that solves the puzzle SEND + MORE = MONEY by giving every letter a different digit from 0 to 9.

Algorithm

1. Make a list of the different letters in the puzzle. Here there are 8: S, E, N, D, M, O, R and Y.
2. Go through every way of giving these letters different digits.
3. Skip any assignment where a leading letter (S or M) is 0, because a number cannot start with zero.
4. Replace the letters in each word with their digits to get actual numbers.
5. Check whether SEND + MORE equals MONEY. The first assignment that works is the answer, so print it.
6. If no assignment works, print that there is no solution.

Experiment 4: Crypt-Arithmetic Problem – Program

```
from itertools import permutations


def solve_cryptarithm(word1, word2, result):
    letters = sorted(set(word1 + word2 + result))
    if len(letters) > 10:
        return None
    first_letters = {word1[0], word2[0], result[0]}

    def to_number(word, mapping):
        return int("".join(str(mapping[ch]) for ch in word))

    for perm in permutations(range(10), len(letters)):
        mapping = dict(zip(letters, perm))
        if any(mapping[ch] == 0 for ch in first_letters):
            continue  # no leading zeros
        if to_number(word1, mapping) + to_number(word2, mapping) == \
                to_number(result, mapping):
            return mapping
    return None


if __name__ == "__main__":
    w1, w2, res = "SEND", "MORE", "MONEY"
    print("Puzzle:", w1, "+", w2, "=", res)
    sol = solve_cryptarithm(w1, w2, res)
    if sol is None:
        print("No solution found.")
    else:
        print("\nLetter assignment:")
        for ch in sorted(sol):
            print(" ", ch, "=", sol[ch])
        n1 = "".join(str(sol[c]) for c in w1)
        n2 = "".join(str(sol[c]) for c in w2)
        n3 = "".join(str(sol[c]) for c in res)
        print("\n ", n1)
        print("+", n2)
        print("-------")
        print(" ", n3)
```

Experiment 4: Crypt-Arithmetic Problem – Output

D. Vishnu Vardhan Reddy - 192325039 - p4_cryptarithm.py

```
$ python3 p4_cryptarithm.py

Puzzle: SEND + MORE = MONEY

Letter assignment:
  D = 7
  E = 5
  M = 1
  N = 6
  O = 0
  R = 8
  S = 9
  Y = 2

  9567
+ 1085
-------
  10652

$
```

Result: The program found S=9, E=5, N=6, D=7, M=1, O=0, R=8, Y=2, which gives 9567 + 1085 = 10652.

Experiment 5: Missionaries and Cannibals Problem

Aim

To write a Python program that takes 3 missionaries and 3 cannibals across a river in a boat that carries at most 2 people, without ever letting the cannibals outnumber the missionaries on either bank. The search is done with BFS.

Algorithm

1. Write each state as (M, C, B): the missionaries and cannibals on the left bank, and the side the boat is on (0 for left, 1 for right). We start at (3, 3, 0) and want to reach (0, 0, 1).
2. The boat can carry 1 missionary, 2 missionaries, 1 cannibal, 2 cannibals, or 1 of each.
3. From a state, try every boat load, moving it in the direction the boat is currently facing.
4. Accept a new state only if the numbers stay between 0 and 3 and, on each bank, the missionaries are either absent or at least as many as the cannibals.
5. Run BFS from the start state, adding each valid, unseen state to the queue.
6. When the goal state turns up, print the list of crossings.

Experiment 5: Missionaries and Cannibals Problem – Program

```
from collections import deque

# State = (missionaries_left, cannibals_left, boat) ; boat: 0 = left, 1 = right
START = (3, 3, 0)
GOAL = (0, 0, 1)
MOVES = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]  # (missionaries, cannibals) in boat


def is_valid(m, c):
    if m < 0 or c < 0 or m > 3 or c > 3:
        return False
    mr, cr = 3 - m, 3 - c
    if m > 0 and c > m:      # left bank: cannibals outnumber missionaries
        return False
    if mr > 0 and cr > mr:   # right bank: cannibals outnumber missionaries
        return False
    return True


def successors(state):
    m, c, boat = state
    for dm, dc in MOVES:
        if boat == 0:
            nm, nc = m - dm, c - dc
        else:
            nm, nc = m + dm, c + dc
        if is_valid(nm, nc):
            yield (nm, nc, 1 - boat), (dm, dc)


def solve():
    queue = deque([(START, [(START, None)])])
    visited = {START}
    while queue:
        state, path = queue.popleft()
        if state == GOAL:
            return path
        for nxt, move in successors(state):
            if nxt not in visited:
                visited.add(nxt)
                queue.append((nxt, path + [(nxt, move)]))
    return None


if __name__ == "__main__":
    path = solve()
    print("Solution found in", len(path) - 1, "crossings\n")
    print("State format: (Missionaries left, Cannibals left, Boat side)\n")
    print("Start :", path[0][0])
    for i, (state, (dm, dc)) in enumerate(path[1:], 1):
        side = "left -> right" if state[2] == 1 else "right -> left"
        print("Step", i, ":", dm, "M,", dc, "C cross", side, "=>", state)
```

Experiment 5: Missionaries and Cannibals Problem – Output

D. Vishnu Vardhan Reddy - 192325039 - p5_missionaries.py

```
$ python3 p5_missionaries.py

Solution found in 11 crossings

State format: (Missionaries left, Cannibals left, Boat side)

Start : (3, 3, 0)
Step 1 : 0 M, 2 C cross left -> right => (3, 1, 1)
Step 2 : 0 M, 1 C cross right -> left => (3, 2, 0)
Step 3 : 0 M, 2 C cross left -> right => (3, 0, 1)
Step 4 : 0 M, 1 C cross right -> left => (3, 1, 0)
Step 5 : 2 M, 0 C cross left -> right => (1, 1, 1)
Step 6 : 1 M, 1 C cross right -> left => (2, 2, 0)
Step 7 : 2 M, 0 C cross left -> right => (0, 2, 1)
Step 8 : 0 M, 1 C cross right -> left => (0, 3, 0)
Step 9 : 0 M, 2 C cross left -> right => (0, 1, 1)
Step 10 : 1 M, 0 C cross right -> left => (1, 1, 0)
Step 11 : 1 M, 1 C cross left -> right => (0, 0, 1)

$
```

Result: All six people reached the other side safely in 11 crossings.